

Assignment 1: The Eight-Puzzle

CS 205: Introduction to Artificial Intelligence

Pranesh Naagamuthu
SID: 862343247
Email: pnaag001@ucr.edu

May 8, 2026

Statement of Originality and Consultations

In completing this assignment, I consulted the Python 3 documentation for standard library usage (specifically the `heapq` and `copy` modules) [1]. All major code and logic, including the search algorithms and heuristic evaluations, are entirely original. No code was shared with or copied from colleagues or online repositories.

1 Introduction

Sliding tile puzzles, such as the 8-puzzle, challenge a player to slide pieces along legal routes to establish a specific end-configuration. The 8-puzzle consists of a 3x3 grid containing tiles numbered 1 through 8, alongside one blank space. A tile adjacent to the blank space can be shifted into it, effectively moving the blank space around the board.

The objective of this project is to implement and empirically compare three different search algorithms to solve the 8-puzzle optimally: Uniform Cost Search, A* Search using the Misplaced Tile heuristic, and A* Search using the Manhattan Distance heuristic.

2 Methodology and Algorithms

The core engine of the solver relies on a priority queue to explore the state space. Each board configuration is represented as a node containing its current state, the path cost $g(n)$ to reach it, and the heuristic estimated cost $h(n)$ to reach the goal.

2.1 Uniform Cost Search

Uniform Cost Search (UCS) expands the node with the lowest path cost $g(n)$. For this implementation, UCS acts equivalently to Breadth-First Search, as every legal move has a uniform step cost of 1. The heuristic function $h(n)$ is strictly hardcoded to 0. While guaranteed to find the optimal path, it is entirely blind and expands uniformly in all directions.

2.2 A* with Misplaced Tile Heuristic

A* search expands nodes based on the evaluation function $f(n) = g(n) + h(n)$. The Misplaced Tile heuristic calculates $h(n)$ by counting the total number of tiles that are not currently in their goal coordinates. The blank tile is excluded from this count.

2.3 A* with Manhattan Distance Heuristic

The Manhattan Distance heuristic is a more informed admissibility metric. Instead of a binary check of whether a tile is in the correct position, it calculates the physical grid distance each tile must travel to reach its target. For a tile at (x_1, y_1) with a goal of (x_2, y_2) , the distance is:

$$h(n) = \sum_{i=1}^8 (|x_{i,current} - x_{i,goal}| + |y_{i,current} - y_{i,goal}|) \quad (1)$$

This provides a much tighter lower bound on the remaining cost than the Misplaced Tile heuristic.

3 Empirical Results

To evaluate the performance of the algorithms, the solver was tested against puzzles of varying depths. As we can see in Table 1, the choice of heuristic drastically impacts the time complexity (measured in nodes expanded) and space complexity (measured in maximum queue size).

Algorithm	Nodes Expanded (Easy)	Nodes Expanded (Medium)	Nodes Expanded (Hard)
Uniform Cost Search	5	22	100700
A* (Misplaced Tile)	2	4	6194
A* (Manhattan)	2	4	588

Table 1: A comparison of nodes expanded across different algorithms and puzzle difficulties.

Algorithm	Max Queue Size (Easy)	Max Queue Size (Medium)	Max Queue Size (Hard)
Uniform Cost Search	6	18	32237
A* (Misplaced Tile)	3	6	3763
A* (Manhattan)	3	6	369

Table 2: Maximum priority queue size observed during the search process.

The data clearly indicates that as the depth of the optimal solution increases, Uniform Cost Search quickly becomes computationally unfeasible. The Misplaced Tile heuristic provides a significant optimization, but the Manhattan Distance heuristic aggressively prunes the search tree, requiring the fewest node expansions.

4 Conclusion

As we have discovered empirically, an informed heuristic drastically reduces both the time and space complexity required to solve sliding tile puzzles. The Manhattan Distance heuristic strictly dominates the Misplaced Tile heuristic because it perfectly accounts for the minimum number of moves required for each tile, rather than treating all misplaced tiles with an equal penalty of 1. Consequently, A* with Manhattan Distance is the fastest and most memory-efficient algorithm among the three tested.

References

- [1] Python Software Foundation. (n.d.). *heapq — Heap queue algorithm*. Python 3 Documentation. Retrieved from <https://docs.python.org/3/library/heapq.html>

Appendix: Source Code

```
1 import heapq
2 import copy
3
4 trivial = [[1, 2, 3],
5            [4, 5, 6],
6            [7, 8, 0]]
7
8 veryEasy = [[1, 2, 3],
9             [4, 5, 6],
10            [7, 0, 8]]
11
12 easy = [[1, 2, 3],
13         [4, 5, 6],
14         [0, 7, 8]]
15
16 medium = [[1, 2, 3],
17           [5, 0, 6],
18           [4, 7, 8]]
19
20 hard = [[8, 7, 1],
21         [6, 0, 2],
22         [5, 4, 3]]
23
24 eight_goal_state = [[1, 2, 3],
25                    [4, 5, 6],
26                    [7, 8, 0]]
27
28 ALGORITHM_NAMES = {
29     1: "Uniform Cost Search",
30     2: "A* with Misplaced Tile Heuristic",
31     3: "A* with Manhattan Distance Heuristic",
32 }
33
34
35 class Node:
36     def __init__(self, state, parent=None, action=None, g=0, h=0):
37         self.state = state
38         self.parent = parent
39         self.action = action
40         self.g = g
41         self.h = h
42
43     @property
44     def f(self):
45         return self.g + self.h
46
47     def __lt__(self, other):
48         return self.f < other.f
49
50
51 def misplaced_tile_heuristic(current_state, goal_state):
```

```

52     h_cost = 0
53     for i in range(len(current_state)):
54         for j in range(len(current_state[i])):
55             if current_state[i][j] != goal_state[i][j] and current_state[i
56                 ][j] != 0:
57                 h_cost += 1
58     return h_cost
59
60 def manhattan_distance_heuristic(current_state, goal_state):
61     h_cost = 0
62     for i in range(len(current_state)):
63         for j in range(len(current_state[i])):
64             if current_state[i][j] == 0:
65                 continue
66             goal_row = (current_state[i][j] - 1) // 3
67             goal_col = (current_state[i][j] - 1) % 3
68             h_cost += abs(i - goal_row) + abs(j - goal_col)
69     return h_cost
70
71
72 def get_heuristic(algorithm_choice):
73     if algorithm_choice == 1:
74         return lambda state, goal: 0
75     elif algorithm_choice == 2:
76         return misplaced_tile_heuristic
77     elif algorithm_choice == 3:
78         return manhattan_distance_heuristic
79     else:
80         raise ValueError(f"Unknown algorithm choice: {algorithm_choice}")
81
82
83 def find_blank(state):
84     for i in range(len(state)):
85         for j in range(len(state[i])):
86             if state[i][j] == 0:
87                 return i, j
88     raise ValueError("No blank tile found in state.")
89
90
91 def get_neighbors(state):
92     row, col = find_blank(state)
93     n = len(state)
94     moves = [
95         ("Up", row - 1, col),
96         ("Down", row + 1, col),
97         ("Left", row, col - 1),
98         ("Right", row, col + 1),
99     ]
100
101     neighbors = []
102     for action, new_row, new_col in moves:
103         if 0 <= new_row < n and 0 <= new_col < n:
104             new_state = copy.deepcopy(state)

```

```

105         new_state[row][col], new_state[new_row][new_col] = (
106             new_state[new_row][new_col],
107             new_state[row][col],
108         )
109         neighbors.append((action, new_state))
110     return neighbors
111
112
113 def traceback(node):
114     path = []
115     current = node
116     while current is not None:
117         path.append(current)
118         current = current.parent
119     path.reverse()
120
121     print("\n=== Solution Path ===")
122     for step, n in enumerate(path):
123         if step == 0:
124             print(f"Initial State:")
125         else:
126             print(f"Step {step} - Move blank {n.action}:  g={n.g}, h={n.h}
127                 }, f={n.f}")
128             for row in n.state:
129                 print(" ", row)
130             print()
131
132 def general_search(initial_state, goal_state, algorithm_choice):
133     heuristic = get_heuristic(algorithm_choice)
134
135     h0 = heuristic(initial_state, goal_state)
136     start_node = Node(state=initial_state, g=0, h=h0)
137
138     priority_queue = []
139     heapq.heappush(priority_queue, start_node)
140
141     visited_states = set()
142     nodes_expanded = 0
143     max_queue_size = 1
144
145     while priority_queue:
146         max_queue_size = max(max_queue_size, len(priority_queue))
147         current_node = heapq.heappop(priority_queue)
148
149         if current_node.state == goal_state:
150             traceback(current_node)
151             print(f"Goal reached!")
152             print(f"  Solution depth   : {current_node.g}")
153             print(f"  Nodes expanded   : {nodes_expanded}")
154             print(f"  Max queue size   : {max_queue_size}")
155             return current_node
156
157     state_tuple = tuple(tuple(row) for row in current_node.state)

```

```

158         if state_tuple in visited_states:
159             continue
160         visited_states.add(state_tuple)
161         nodes_expanded += 1
162
163         for action, new_state in get_neighbors(current_node.state):
164             child_state_tuple = tuple(tuple(row) for row in new_state)
165             if child_state_tuple in visited_states:
166                 continue
167
168             new_g = current_node.g + 1
169             new_h = heuristic(new_state, goal_state)
170             child_node = Node(
171                 state=new_state,
172                 parent=current_node,
173                 action=action,
174                 g=new_g,
175                 h=new_h,
176             )
177             heapq.heappush(priority_queue, child_node)
178
179         print("Failure: Queue is empty and goal was not found.")
180         return None
181
182
183 # CLI / DRIVER
184
185 def read_puzzle():
186     print("Enter your puzzle row by row, using space-separated integers.")
187     print("Use 0 to represent the blank tile.")
188     state = []
189     for i in range(3):
190         while True:
191             raw = input(f"    Row {i + 1}: ").strip().split()
192             if len(raw) == 3 and all(r.isdigit() for r in raw):
193                 state.append([int(r) for r in raw])
194                 break
195             print("    Invalid input - enter exactly 3 integers separated by spaces.")
196     return state
197
198
199 def main():
200     print("=" * 50)
201     print("8-Puzzle Solver")
202     print("=" * 50)
203
204     print("\nSelect puzzle:")
205     print("    1. Use a default puzzle")
206     print("    2. Enter your own puzzle")
207     puzzle_choice = input("Choice: ").strip()
208
209     if puzzle_choice == "1":
210         defaults = {

```

```

211         "1": ("Trivial", trivial),
212         "2": ("Very Easy", veryEasy),
213         "3": ("Easy", easy),
214         "4": ("Medium", medium),
215         "5": ("Hard", hard)
216     }
217     print("\nDefault puzzles:")
218     for k, (name, _) in defaults.items():
219         print(f" {k}. {name}")
220     sel = input("Choice: ").strip()
221     if sel not in defaults:
222         print("Invalid selection - using Trivial.")
223         sel = "1"
224     label, initial_state = defaults[sel]
225     print(f"\nUsing '{label}' puzzle:")
226 else:
227     initial_state = read_puzzle()
228     print("\nYour puzzle:")
229
230 for row in initial_state:
231     print(" ", row)
232
233 print("\nSelect algorithm:")
234 for k, name in ALGORITHM_NAMES.items():
235     print(f" {k}. {name}")
236 while True:
237     alg = input("Choice: ").strip()
238     if alg in {"1", "2", "3"}:
239         algorithm_choice = int(alg)
240         break
241     print(" Please enter 1, 2, or 3.")
242
243 print(f"\nRunning {ALGORITHM_NAMES[algorithm_choice]}\n")
244 general_search(initial_state, eight_goal_state, algorithm_choice)
245
246
247 if __name__ == "__main__":
248     main()

```